

Features of Tuples in Python

A **tuple** is an **immutable**, **ordered**, and **heterogeneous** data structure in Python. It is commonly used to group related data into a single compound data type.

Key Features of Tuples

1. Ordered

- Tuples maintain the order of elements. Once created, the order does not change.
- Example:

```
my_tuple = (10, 20, 30)
print(my_tuple[1]) # Output: 20
```

2. Immutable

- Tuples cannot be modified after creation. You cannot add, remove, or change elements in a tuple.
- Example:

```
my_tuple = (1, 2, 3)
# my_tuple[1] = 20 # This will raise a TypeError
```

3. Heterogeneous

- Tuples can store elements of different data types, such as integers, strings, floats, and even other tuples.
- Example:

```
my_tuple = (1, "hello", 3.14, [1, 2])
print(my_tuple) # Output: (1, "hello", 3.14, [1, 2])
```

4. Supports Nesting

- Tuples can contain other tuples or any other collection type (e.g., lists or dictionaries).
- Example:

```
nested_tuple = ((1, 2), (3, 4))
print(nested_tuple[1]) # Output: (3, 4)
```

5. Indexed

- Tuples use **zero-based indexing** to access their elements.
- Example:

```
my_tuple = (10, 20, 30)
print(my_tuple[0]) # Output: 10
print(my_tuple[-1]) # Output: 30 (negative indexing)
```

6. Iterable

- Tuples can be iterated over using loops or comprehensions.
- Example:

```
my_tuple = (1, 2, 3)
for item in my_tuple:
    print(item)
```

7. Hashable

- Tuples are hashable, meaning they can be used as keys in dictionaries or elements in sets (only if all elements within the tuple are hashable).
- Example:

```
my_dict = {(1, 2): "value"}
print(my_dict[(1, 2)]) # Output: "value"
```

8. Compact and Memory-Efficient

- Tuples require less memory compared to lists, making them faster and more efficient for fixed-size data collections.

9. Supports Slicing

- Tuples support slicing operations to access subparts of the tuple.
- Example:

```
my_tuple = (10, 20, 30, 40, 50)
print(my_tuple[1:4]) # Output: (20, 30, 40)
```

10. Cannot Be Resized

- Since tuples are immutable, you cannot change their size (add or remove elements).

11. Used for Fixed Data

- Tuples are often used to represent fixed collections of items, such as coordinates, dates, or other constant data.

12. Faster Than Lists

- Because of immutability, tuples are faster to process than lists.

13. Supports Methods

- Tuples have only a few built-in methods since they are immutable:
 - `count()`: Counts occurrences of a specific element.
 - `index()`: Finds the index of a specific element.
- Example:

```
my_tuple = (1, 2, 2, 3)
print(my_tuple.count(2)) # Output: 2
print(my_tuple.index(3)) # Output: 3
```

14. Can Contain Duplicate Elements

- Tuples allow duplicate elements, just like lists.
- Example:

```
my_tuple = (1, 2, 2, 3)
print(my_tuple) # Output: (1, 2, 2, 3)
```

15. Parentheses Are Optional

- While tuples are typically created using parentheses `()`, they can also be created without them using a comma-separated list.
- Example:

```
my_tuple = 1, 2, 3 # Tuple without parentheses
print(my_tuple) # Output: (1, 2, 3)
```

Advantages of Tuples

1. **Immutability:** Prevents accidental modification of data, ensuring safety in programs.
2. **Performance:** Faster than lists due to their immutability.
3. **Hashable:** Can be used as dictionary keys or in sets.
4. **Lightweight:** Memory-efficient compared to lists.

In Python, an **element is hashable** if it has a fixed hash value that does not change during its lifetime. This means it can be used as a **key in dictionaries** or **an element in sets**, which require elements to be uniquely identifiable.

• Hash Function

- Hashable elements have a hash value generated by the `hash()` function.
- The hash value is an integer that serves as a unique identifier for the element in hash-based data structures like sets and dictionaries.

Example:

```
print(hash(42)) # Output: A unique integer (e.g., 42)
print(hash("hello")) # Output: A unique integer (e.g., -
6917561972071111893)
```

• Immutability

- For an element to be hashable, it must be **immutable**. This ensures that its hash value remains constant.
- Examples of immutable types:
 - Numbers: `int`, `float`
 - Strings: `str`
 - Tuples (if all elements inside are hashable)
- Examples of non-hashable types:
 - Lists: `list` (mutable)
 - Dictionaries: `dict` (mutable)
 - Sets: `set` (mutable)

Here are the **operations you can perform on tuples** in Python, with examples:

1. Accessing Elements

a. Indexing

Access elements in a tuple using zero-based indexing.

```
my_tuple = (10, 20, 30, 40)
print(my_tuple[1])  # Output: 20
print(my_tuple[-1]) # Output: 40 (last element)
```

b. Slicing

Retrieve a subset of the tuple using slicing.

```
print(my_tuple[1:3]) # Output: (20, 30)
print(my_tuple[:3])  # Output: (10, 20, 30)
print(my_tuple[::-1]) # Output: (40, 30, 20, 10) (reversed)
```

2. Iterating Over Tuples

You can loop through a tuple using a `for` loop.

```
for item in my_tuple:
    print(item)
# Output:
# 10
# 20
# 30
# 40
```

3. Concatenation

Combine two or more tuples using the `+` operator.

```
tuple1 = (1, 2)
tuple2 = (3, 4)
result = tuple1 + tuple2
print(result)  # Output: (1, 2, 3, 4)
```

4. Repetition

Repeat a tuple using the `*` operator.

```
my_tuple = (1, 2)
print(my_tuple * 3)  # Output: (1, 2, 1, 2, 1, 2)
```

5. Membership Testing

Check if an element exists in a tuple using the `in` operator.

```
my_tuple = (10, 20, 30)
print(20 in my_tuple)  # Output: True
print(50 in my_tuple)  # Output: False
```

6. Finding the Length

Use the `len()` function to find the number of elements in a tuple.

```
my_tuple = (10, 20, 30)
print(len(my_tuple))  # Output: 3
```

7. Counting Elements

Use the `count()` method to count the occurrences of a value in a tuple.

```
my_tuple = (10, 20, 20, 30)
print(my_tuple.count(20))  # Output: 2
```

8. Finding the Index

Use the `index()` method to find the first occurrence of a value in a tuple.

```
my_tuple = (10, 20, 30, 20)
print(my_tuple.index(20))  # Output: 1
```

9. Tuple Packing and Unpacking

a. Packing

Group multiple values into a tuple.

```
my_tuple = 1, 2, 3  # Parentheses are optional
print(my_tuple)  # Output: (1, 2, 3)
```

b. Unpacking

Assign tuple values to multiple variables.

```
a, b, c = (1, 2, 3)
print(a)   # Output: 1
print(b)   # Output: 2
print(c)   # Output: 3
```

10. Nested Tuples

Tuples can contain other tuples as elements.

```
nested_tuple = ((1, 2), (3, 4), (5, 6))
print(nested_tuple[1])   # Output: (3, 4)
print(nested_tuple[1][0]) # Output: 3
```

11. Maximum, Minimum, and Sum

Perform mathematical operations on tuples containing numbers.

```
my_tuple = (10, 20, 30)
print(max(my_tuple)) # Output: 30
print(min(my_tuple)) # Output: 10
print(sum(my_tuple)) # Output: 60
```

12. Sorting a Tuple

Sort the elements of a tuple. Since tuples are immutable, you need to convert them into a list first.

```
my_tuple = (30, 10, 20)
sorted_tuple = tuple(sorted(my_tuple))
print(sorted_tuple) # Output: (10, 20, 30)
```

13. Immutable Property

Tuples are immutable, so you cannot modify, add, or remove elements. Attempting to do so raises an error.

```
my_tuple = (10, 20, 30)
# my_tuple[1] = 100 # Raises TypeError: 'tuple' object does not support
# item assignment
```

14. Creating a Single-Element Tuple

To create a tuple with one element, you must include a trailing comma.

```
single_element_tuple = (42,)
print(type(single_element_tuple))  # Output: <class 'tuple'>
```

15. Tuple as Dictionary Keys

Tuples are hashable and can be used as dictionary keys (if they contain only hashable elements).

```
my_dict = {(1, 2): "value"}
print(my_dict[(1, 2)])  # Output: "value"
```

16. Joining Tuples

Use the `*` operator to join tuples inside another tuple using unpacking.

```
tuple1 = (1, 2, 3)
tuple2 = (4, 5)
combined = (*tuple1, *tuple2)
print(combined)  # Output: (1, 2, 3, 4, 5)
```

17. Convert List to Tuple (and Vice Versa)

a. Convert a List to a Tuple

```
my_list = [10, 20, 30]
my_tuple = tuple(my_list)
print(my_tuple)  # Output: (10, 20, 30)
```

b. Convert a Tuple to a List

```
my_tuple = (10, 20, 30)
my_list = list(my_tuple)
print(my_list)  # Output: [10, 20, 30]
```